LAPORAN PRAKTIKUM

PEMROGRAMAN BERORIENTASI OBJEK

MODUL IV & V

OOP AND ENCAPSULATION

Disusun Oleh:

Silviana Maristha Ginting 105224001

PROGRAM STUDI ILMU KOMPUTER

FAKULTAS SAINS DAN KOMPUTER

UNIVERSITAS PERTAMINA

2026

I.Pendahuluan

Program ini dibuat untuk mensimulasikan sistem pemesanan kamar hotel menggunakan bahasa pemrograman Java dengan konsep Object-Oriented Programming (OOP). Program ini bertujuan untuk mengelola data kamar seperti nomor kamar, tipe kamar, kapasitas maksimal, harga per malam, serta status ketersediaan kamar. Selain itu, program juga menyediakan fitur pemesanan, pembatalan, serta perhitungan total biaya menginap dengan atau tanpa diskon. Dengan adanya program ini, proses pengelolaan kamar menjadi lebih terstruktur dan mudah dikontrol.

II. Variabel

No	Nama Variabel	Tipe Data	Fungsi
1	nomorKamar	String	Menyimpan nomor kamar
2	tipeKamar	String	Menyimpan jenis kamar (Reguler, Premium, Suite)
3	kapasitasMaksimal	int	Menentukan jumlah maksimal tamu
4	hargaPerMalam	double	Menyimpan harga sewa per malam
5	tersedia	boolean	Menentukan status kamar (tersedia atau terisi)

III. Constructor dan Method

No	Nama Metode / Constructor	Jenis Metode	Fungsi
1	kamar()	Constructor	Menginisialisasi data awal kamar
2	setKamarHotel()	Procedural	Mengatur ulang seluruh data kamar
3	getNomorKamar()	Functional	Mengambil nomor kamar
4	getTipeKamar()	Functional	Mengambil tipe kamar
5	getKapasitasMaksimal()	Functional	Mengambil kapasitas maksimal
6	getHargaPerMalam()	Functional	Mengambil harga per malam
7	isTersedia()	Functional	Mengecek status kamar
8	setTipeKamar()	Procedural	Validasi tipe kamar
9	setHargaPerMalam()	Procedural	Validasi harga minimum
10	pesanKamar()	Procedural	Memesan kamar tanpa cek tamu
11	pesanKamar(int jumlahTamu)	Procedural (Overloading)	Memesan dengan cek kapasitas
12	batalPesan()	Procedural	Membatalkan pesanan
13	hitungTotalBayar(int)	Functional	Hitung total tanpa diskon
14	hitungTotalBayar(int, String)	Functional (Overloading)	Hitung total dengan diskon

IV. Dokumentasi dan Pembahasan Code

1. Program ini menggunakan konsep Object-Oriented Programming (OOP) dengan menerapkan encapsulation melalui penggunaan atribut private. Atribut seperti nomorKamar, tipeKamar, kapasitasMaksimal, hargaPerMalam, dan tersedia hanya dapat diakses melalui method getter dan setter sehingga data lebih aman dan terkontrol.

```java
class kamar {
    private String nomorKamar;
    private String tipeKamar;
    private int kapasitasMaksimal;
    private double hargaPerMalam;
    private boolean tersedia;
```

2. Constructor kamar() digunakan untuk menginisialisasi data awal kamar saat objek dibuat. Pada constructor ini, tipe kamar langsung divalidasi menggunakan method setTipeKamar() agar sesuai aturan. Selain itu, harga awal diset 0 dan status kamar langsung tersedia. Method setKamarHotel() digunakan untuk mengatur ulang seluruh data kamar. Method ini tetap memanfaatkan setter agar setiap input melalui proses validasi.

```java
    // constructor cepat
    public kamar(String nomorKamar, String tipeKamar, int kapasitasMaksimal) {
        this.nomorKamar = nomorKamar;
        setTipeKamar(tipeKamar);
        this.kapasitasMaksimal = kapasitasMaksimal;
        this.hargaPerMalam = 0;
        this.tersedia = true;
    }

    // constructor lengkap
    public void setKamarHotel(String nomorKamar, String tipeKamar, int kapasitasMaksimal, double hargaPerMalam) {
        this.nomorKamar = nomorKamar;
        setTipeKamar(tipeKamar);
        this.kapasitasMaksimal = kapasitasMaksimal;
        setHargaPerMalam(hargaPerMalam);
        this.tersedia = true;
    }
```

3. Method getter digunakan untuk mengambil nilai atribut yang bersifat private agar tetap bisa diakses dari luar class. Pada kode ini, getNomorKamar(), getTipeKamar(), getKapasitasMaksimal(), dan getHargaPerMalam() berfungsi untuk mengembalikan data kamar, sedangkan isTersedia() digunakan untuk mengecek status kamar. Getter ini merupakan bagian dari konsep encapsulation agar data tetap aman.

```
// getter
public String getNomorKamar() {
return nomorKamar;
}

public String getTipeKamar() {
return tipeKamar;
}

public int getKapasitasMaksimal() {
return kapasitasMaksimal;
}

public double getHargaPerMalam() {
return hargaPerMalam;
}

public boolean isTersedia() {
return tersedia;
}
```

4. Method setTipeKamar() berfungsi untuk mengatur tipe kamar dengan validasi. Hanya tipe Reguler, Premium, dan Suite yang diperbolehkan. Jika tidak sesuai, maka otomatis diubah menjadi Reguler.

```
public void setTipeKamar(String tipeKamar) {
    if (tipeKamar.equals(anObject: "Reguler") || tipeKamar.equals(anObject: "Premium") || tipeKamar.equals(anObje
        this.tipeKamar = tipeKamar;
    } else {
        System.out.println(x: "Tipe tidak valid, diubah ke Reguler");
        this.tipeKamar = "Reguler";
    }
}
```

5. Method setHargaPerMalam() digunakan untuk mengatur harga kamar dengan batas minimum 50.000.

```java
public void setHargaPerMalam(double hargaPerMalam) {
    if (hargaPerMalam < 50000) {
        System.out.println(x: "Harga terlalu rendah, diubah ke 50000");
        this.hargaPerMalam = 50000;
    } else {
        this.hargaPerMalam = hargaPerMalam;
    }
}
```

6. Method pesanKamar() digunakan untuk melakukan pemesanan kamar tanpa mengecek jumlah tamu. Jika kamar tersedia, maka status berubah menjadi terisi. Jika tidak, maka akan muncul pesan bahwa kamar sudah terisi.

```java
public void pesanKamar() {
    if (tersedia) {
        tersedia = false;
        System.out.println(x: "Kamar berhasil dipesan");
    } else {
        System.out.println(x: "Kamar sudah terisi");
    }
}
```

7. Method pesanKamar(int jumlahTamu) merupakan method overloading yang menambahkan pengecekan kapasitas. Jika jumlah tamu melebihi kapasitas atau kamar sudah terisi, maka pemesanan ditolak.

```java
public void pesanKamar(int jumlahTamu) {
    if (!tersedia) {
        System.out.println(x: "Kamar sudah terisi");
    } else if (jumlahTamu > kapasitasMaksimal) {
        System.out.println(x: "Jumlah tamu melebihi kapasitas");
    } else {
        tersedia = false;
        System.out.println(x: "Pemesanan berhasil");
    }
}
```

8. Method batalPesan() berfungsi untuk membatalkan pemesanan dan mengubah status kamar menjadi tersedia kembali.

```java
public void batalPesan() {
    tersedia = true;
    System.out.println(x: "Pesanan dibatalkan");
}
```

9. Method hitungTotalBayar(int jumlahMalam) digunakan untuk menghitung total biaya tanpa diskon, yaitu jumlah malam dikalikan harga per malam.

```java
public double hitungTotalBayar(int jumlahMalam) {
    return jumlahMalam * hargaPerMalam;
}
```

10. Method hitungTotalBayar(int jumlahMalam, String kodeVoucher) merupakan overloading yang menambahkan fitur diskon. Jika menggunakan kode "PROMO" dan menginap minimal 3 malam, maka diberikan diskon 20%. Jika tidak, maka harga tetap normal.

```java
public double hitungTotalBayar(int jumlahMalam, String kodeVoucher) {
    double total = jumlahMalam * hargaPerMalam;

    if (kodeVoucher.equals(anObject: "PROMO") && jumlahMalam >= 3) {
        System.out.println(x: "Diskon 20% diterapkan");
        return total * 0.8;
    } else {
        System.out.println(x: "Voucher tidak berlaku");
        return total;
    }
}
```

11. Pada class App, method main() digunakan untuk menguji seluruh fitur program. Program membuat dua objek kamar, mengatur harga, menampilkan informasi, melakukan pemesanan, serta menghitung total biaya. Hasil output menunjukkan bahwa seluruh fungsi berjalan sesuai dengan logika yang telah dibuat.

```java
public class App {
    Run | Debug
    public static void main(String[] args) {


        kamar kamar1 = new kamar(nomorKamar: "101", tipeKamar: "Reguler", kapasitasMaksimal: 2);
         kamar1.setHargaPerMalam(hargaPerMalam: 300000);


        kamar kamar2 = new kamar(nomorKamar: "102", tipeKamar: "Premium", kapasitasMaksimal: 2);
        kamar2.setHargaPerMalam(hargaPerMalam: 500000);

        System.out.println(x: "=== UJI SETTER ===");
        System.out.println(x: "=== Informasi Kamar ===");
        System.out.println("Nomor Kamar   : " + kamar1.getNomorKamar());
        System.out.println("Tipe Kamar    : " + kamar1.getTipeKamar());
        System.out.println("Kapasitas     : " + kamar1.getKapasitasMaksimal());
        System.out.println("Harga/Malam   : " + kamar1.getHargaPerMalam());
        System.out.println("Status        : " + (kamar1.isTersedia() ? "Tersedia" : "Terisi"));
        System.out.println();

        System.out.println(x: "=== UJI PEMESANAN ===");
        kamar2.pesanKamar(jumlahTamu: 4);
        kamar2.pesanKamar(jumlahTamu: 2);
        kamar2.pesanKamar();

        System.out.println(x: "\n=== UJI PEMBAYARAN ===");
        double total1 = kamar1.hitungTotalBayar(jumlahMalam: 2, kodeVoucher: "PROMO");
        System.out.println("Total kamar1: " + total1);

        double total2 = kamar2.hitungTotalBayar(jumlahMalam: 4, kodeVoucher: "PROMO");
        System.out.println("Total kamar2: " + total2);

        System.out.println(x: "\n=== STRUK AKHIR ===");
        System.out.println("Kamar 1: " + kamar1.getNomorKamar() + " - " + kamar1.getTipeKamar() + " - Total: " + total1);
        System.out.println("Kamar 2: " + kamar2.getNomorKamar() + " - " + kamar2.getTipeKamar() + " - Total: " + total2);

    }
}
```

IV. Kesimpulan

Program ini berhasil mengimplementasikan konsep dasar Object-Oriented Programming (OOP) dalam sistem pemesanan kamar hotel. Penggunaan encapsulation membuat data lebih aman, sedangkan method overloading memberikan fleksibilitas dalam penggunaan fungsi. Program juga dilengkapi dengan validasi input dan fitur diskon sehingga lebih realistis. Secara keseluruhan, program berjalan dengan baik dan dapat dikembangkan lebih lanjut menjadi sistem yang lebih kompleks.

IV. Daftar Pustaka

Praktikum.